

React Project에 CI/CD적용하기

시나리오

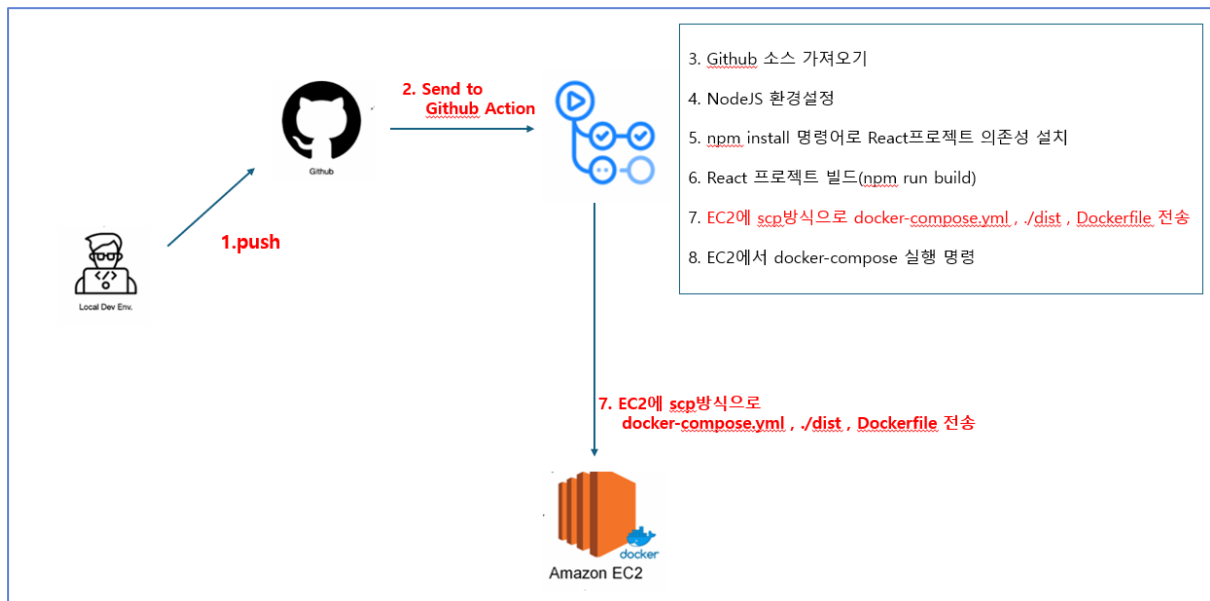
첫번째,

Nginx 웹서버는 Dockerfile을 이용하여 이미지를 생성하고 AWS EC2에 Docker기반으로 docker-compose.yml 을 이용하여 실행 시킨다.

두번째,

GitHub secrets에 민감한 정보를 저장하고 설정문서에서 사용하도록 한다.

전체흐름



작업순서

1. EC2생성 – front_react (t3.small , port : 80 , 22 , 443)
2. EC2에 Docker 설치
3. github에 새로운 프로젝트 생성(cicd03_front_react) 하고 boot 프로젝트 연결
 - .gitignore에서 dist부분 주석 풀기
4. github > Settings > Secrets and Variables > Actions > New repository secret등록

6.React프로젝트에 설정문서 만들기 및 수정

- Dockerfile
- docker-compose.yml
- .github/workflows/deploy.yml

EC2생성

① EC2에서 인스턴스를 처음 시작하시는 것 같습니다. 이 안내를 통해 EC2, 인스턴스 시작 방법 및 모범 사례에 대해 알아보세요.

인스턴스 시작 정보

Amazon EC2를 사용하면 AWS 클라우드에서 실행되는 가상 머신 또는 인스턴스를 생성할 수 있습니다. 아래의 간단한 단계에 따라 빠르게 시작할 수 있습니다.

이름 및 태그 정보

이름

front_react

추가 태그 추가

▼ 애플리케이션 및 OS 이미지(Amazon Machine Image) 정보

AMI는 인스턴스를 시작하는 데 필요한 소프트웨어 구성(운영 체제, 애플리케이션 서버 및 애플리케이션)이 포함된 템플릿입니다. 아래에서 찾고 있는 항목이 보이지 않으면 AMI를 검색하거나 찾아보세요.

Q 수천 개의 애플리케이션 및 OS 이미지를 포함하는 전체 카탈로그 검색

최근 사용

Quick Start

Amazon Linux

macOS

Ubuntu

Windows

Red Hat

SUSE Linux

Debian

aws

Mac

ubuntu

Microsoft

Red Hat

SUSE

debian

더 많은 AMI 찾아보기

AWS, Marketplace 및 커뮤니티의 AMI 포함

Amazon Machine Image(AMI)

Ubuntu Server 24.04 LTS (HVM), SSD Volume Type

ami-0d5bb3742db8fc264 (64비트(x86)) / ami-064b0e4b6529b20cf (64비트(Arm))

가상화: hvm ENA 활성화됨: true 루트 디바이스 유형: ebs

프리 티어 사용 가능 ▼

설명

Ubuntu Server 24.04 LTS (HVM),EBS General Purpose (SSD) Volume Type. Support available from Canonical (<http://www.ubuntu.com/cloud/services>).

Canonical, Ubuntu, 24.04, amd64 noble image

아키텍처

AMI ID

계시 날짜

사용자 이름

①

64비트(x86) ▼

ami-0d5bb3742db8fc264

2025-03-05

ubuntu

확인된 공급 업체

▼ 인스턴스 유형 정보 | [조언 받기](#)

인스턴스 유형

t3.small

패밀리: t3

2 vCPU

2 GiB 메모리

현재 세대: true

온디맨드 Ubuntu Pro 기본 요금: 0.0295 USD 시간당

온디맨드 RHEL 기본 요금: 0.0548 USD 시간당

온디맨드 Linux 기본 요금: 0.026 USD 시간당

온디맨드 Windows 기본 요금: 0.0444 USD 시간당

온디맨드 SUSE 기본 요금: 0.057 USD 시간당

모든 세대

인스턴스 유형 비교

소프트웨어가 사전 설치된 AMI에는 추가 비용이 적용됩니다.

▼ 키 페어(로그인) 정보

키 페어를 사용하여 인스턴스에 안전하게 연결할 수 있습니다. 인스턴스를 시작하기 전에 선택한 키 페어에 대한 액세스 권한이 있는지 확인하세요.

키 페어 이름 - 필수

my-key

새 키 페어 생성

▼ 네트워크 설정 정보

네트워크 | 정보

vpc-029bf3f33d1054705

서브넷 | 정보

기본 설정 없음(가용 영역의 기본 서브넷)

퍼블릭 IP 자동 할당 | 정보

할당자

프리 티어 허용 범위를 벗어나는 경우 추가 요금이 적용됩니다.

방화벽(보안 그룹) | 정보

보안 그룹은 인스턴스에 대한 트래픽을 제어하는 방화벽 규칙 세트입니다. 특정 트래커이 인스턴스에 도달하도록 허용하는 규칙을 추가합니다.

보안 그룹 생성

기존 보안 그룹 선택

다음 규칙을 사용하여 'launch-wizard-4'이라는 새 보안 그룹을 생성합니다.

다음에서 SSH 트래픽 허용

인스턴스 연결에 도움

인터넷에서 HTTPS 트래픽 허용

예를 들어 웹 서버를 생성할 때 엔드포인트를 설정하려면

인터넷에서 HTTP 트래픽 허용

예를 들어 웹 서버를 생성할 때 엔드포인트를 설정하려면

위치 무관

0.0.0.0/0

소스가 0.0.0.0/0인 규칙은 모든 IP 주소에서 인스턴스에 액세스하도록 허용합니다. 알려진 IP 주소의 액세스만 허용하도록 보안 그룹을 설정하는 것이 좋습니다.

▼ 스토리지 구성 정보

1x 30 GiB gp3

루트 볼륨, 3000IOPS, 암호화되지 않음

프리 티어를 사용할 수 있는 고객은 최대 30GB의 EBS 볼륨(SSD) 또는 마그네틱 스토리지를 사용할 수 있습니다.

새 볼륨 추가

소프트웨어 이미지(AMI)

Canonical, Ubuntu, 24.04, amd64...더 보기

ami-0d5b63742db8fc264

가상 서버 유형(인스턴스 유형)

t3.small

방화벽(보안 그룹)

새 보안 그룹

스토리지(볼륨)

1개의 볼륨 - 30GiB

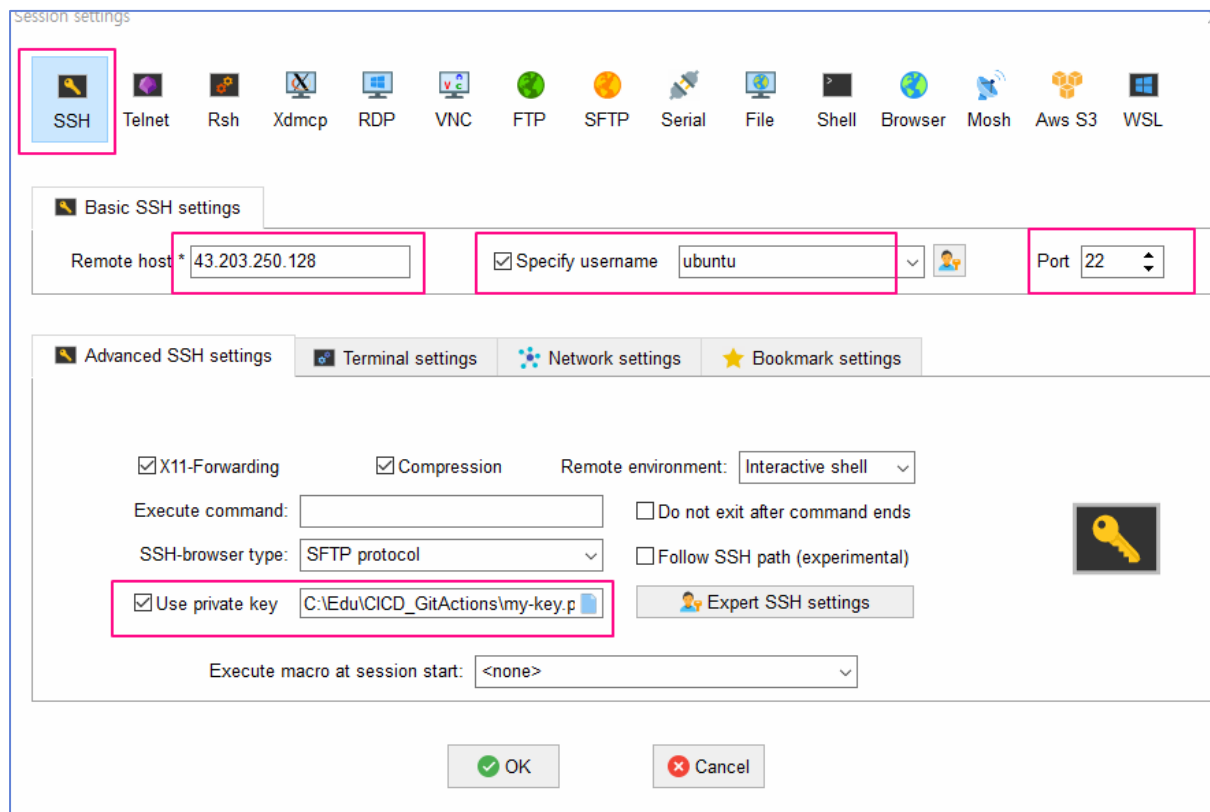
프리 티어: AWS 계정을 개설한 첫해에 프리 티어 AMI, 매월 750시간의 퍼블릭 IPv4 주소 사용량, 30GiB의 EBS 스토리지, 2백만 I/O, 1GB의 스냅샷 및 100GB의 인터넷 대역폭과 함께 사용하면 매월 750시간의 t2.micro 인스턴스 사용량(또는 t2.micro를 사용할 수 없는 경우 t3.micro)을 제공합니다.

취소

인스턴스 시작

미리 보기 코드

Mobxterm접속



Docker 설치

```

sudo apt-get update && ₩
    sudo apt-get install -y apt-transport-https ca-certificates curl software-properties-
common && ₩
    curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add - && ₩
    sudo apt-key fingerprint 0EBFCD88 && ₩
    sudo add-apt-repository "deb [arch=amd64] https://download.docker.com/linux/ubuntu
$(lsb_release -cs) stable" && ₩
    sudo apt-get update && ₩
    sudo apt-get install -y docker-ce && ₩
    sudo usermod -aG docker ubuntu && ₩
    sudo curl -L "https://github.com/docker/compose/releases/download/1.23.2/docker-
compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose && ₩
    sudo chmod +x /usr/local/bin/docker-compose && ₩
    sudo ln -s /usr/local/bin/docker-compose /usr/bin/docker-compose
  
```

```

100 11.2M 100 11.2M 0 0 9751k 0 0:00:01 0:00:01 --:--:-- 211M
ubuntu@ip-172-31-14-254:~$ docker --version
Docker version 28.1.1, build 4eba377
ubuntu@ip-172-31-14-254:~$

```

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere?
[Import a repository.](#)

Required fields are marked with an asterisk (*).

Owner *  8253jang / Repository name *

 cicd03_front_react is available.

Great repository names are short and memorable. Need inspiration? How about [expert-disco](#) ?

Description (optional)

- ☒  **Public**
 Anyone on the internet can see this repository. You choose who can commit.
- ☐  **Private**
 You choose who can see and commit to this repository.

```

KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/AWS/step20-jwt-aws
$ git init
Initialized empty Git repository in D:/Edu_286/AWS/step20-jwt-aws/.git/

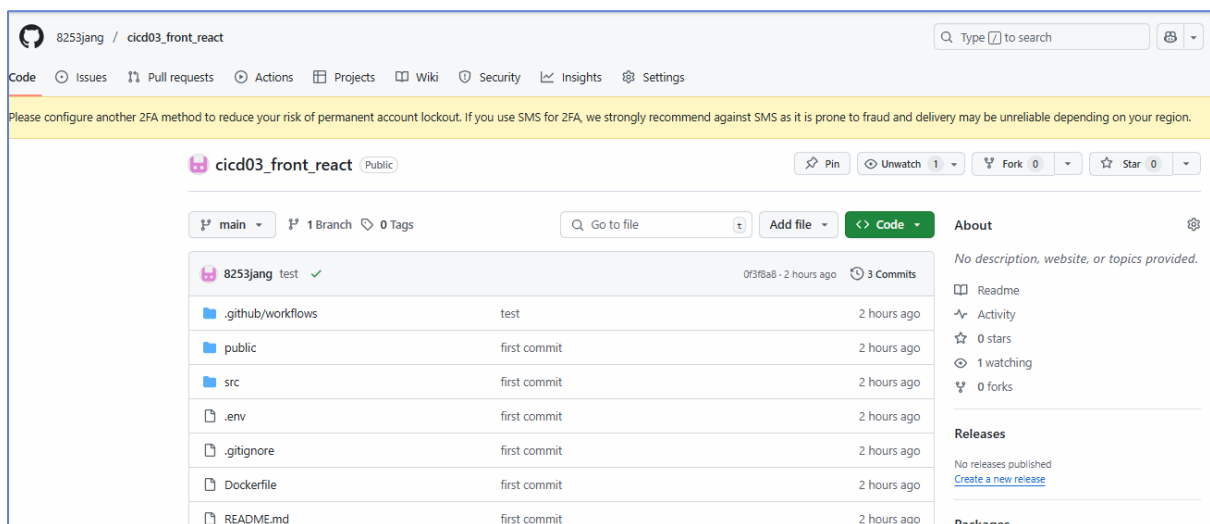
KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/AWS/step20-jwt-aws (main)
$ git add .

KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/AWS/step20-jwt-aws (main)
$ git commit -m "first commit"
[main (root-commit) 7ff6420] first commit
42 files changed, 5701 insertions(+)
create mode 100644 .env
create mode 100644 .github/workflows/deploy.yml

```

```
KOSTA@DESKTOP-U2D32M7 MTNGW64 /d/Edu_286/AWS/step20-jwt-aws (main)
$ git remote add origin https://github.com/8253jang/cicd03_front_react.git

KOSTA@DESKTOP-U2D32M7 MTNGW64 /d/Edu_286/AWS/step20-jwt-aws (main)
$ git push origin main
Enumerating objects: 53, done.
Counting objects: 100% (53/53), done.
Delta compression using up to 8 threads
Compressing objects: 100% (50/50), done.
Writing objects: 100% (53/53), 1.38 MiB | 1.74 MiB/s, done.
Total 53 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), done.
To https://github.com/8253jang/cicd03_front_react.git
* [new branch]      main -> main
```

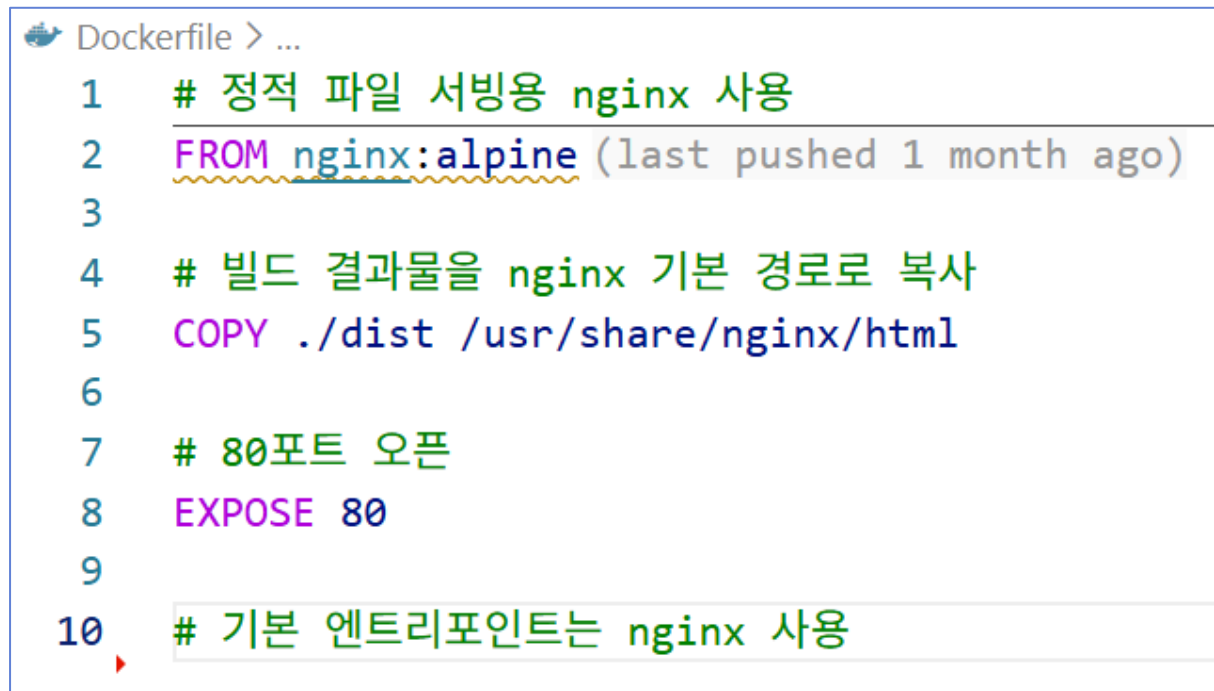


GitHub Secrets 목록(3개)

이름	설명
EC2_HOST	EC2 퍼블릭 IP
EC2_USERNAME	ubuntu
EC2_SSH_KEY	.pem 파일의 텍스트



Dockerfile 작성



alpine은 **Alpine Linux**를 기반으로 하는 경량 리눅스 배포판

nginx:alpine 뜻

- nginx: Nginx 웹 서버를 포함한 도커 이미지
- alpine: Alpine Linux 기반으로 제작된 이미지 (초경량)

이미지 이름	크기	특징
nginx:latest	약 133MB	Debian 기반, 다양한 기능
nginx:alpine	약 5MB	Alpine 기반, 경량화된 구성

nginx:alpine은 가볍고 빠른 nginx 웹 서버 이미지로,

정적 파일 서빙이나 단순 웹 서비스 배포에 매우 적합하다.

docker-compose.yml 작성

```

🐳 docker-compose.yml > ...
    docker-compose.yml - The Compose specification establishes a standard for the definition of m
1  services:                # 여러 서비스(컨테이너) 정의 시작
    ▶ Run Service
2  |   react-app:           # 서비스 이름 (사용자 정의 이름)
3  |     build: .            # 현재 디렉토리의 Dockerfile을 기준으로 이미지 빌드
4  |     container_name: react-container # 생성될 컨테이너의 이름 지정
5  |     ports:
6  |       - "80:80"        # 호스트 포트:컨테이너 포트 매핑
7  |     restart: always    # 컨테이너가 중지되면 항상 재시작
8

```

.github/workflows/deploy.yml

```

1  name: Deploy React App to EC2 # 이 GitHub Actions 워크플로우의 이름 (Actions 탭에 표시됨)
2
3  on:
4  |   push:
5  |     branches: [ main ]      # main 브랜치에 push가 발생할 때 이 워크플로우 실행
6
7  jobs:
8  |   deploy:                  # 배포 작업의 고유 ID (원하면 다른 이름 사용 가능)
9  |     name: Deploy to EC2    # GitHub Actions UI에 표시될 작업 이름
10 |     runs-on: ubuntu-latest  # GitHub Actions가 실행될 가상 머신 (Ubuntu 최신 버전)
11
12 |     steps:
13 |       - name: Checkout code
14 |         uses: actions/checkout@v4 # 현재 레포지토리의 코드를 가져옴 (Git clone)
15
16 |       - name: Setup Node
17 |         uses: actions/setup-node@v4 # Node.js 환경 설정
18 |         with:
19 |           node-version: 20        # Node.js 20버전 설치
20
21 |       - name: Install dependencies
22 |         run: npm install          # React 프로젝트의 의존성 설치
23
24 |       - name: Build React App
25 |         run: npm run build        # 정적 파일로 빌드 (보통 ./dist 또는 ./build 폴더 생성)

```

```

27 - name: Copy docker-compose and build folder to EC2
28   uses: appleboy/scp-action@v0.1.7 # EC2에 파일을 복사(scps)
29   with:
30     host: ${secrets.EC2_HOST} # EC2의 공인 IP 또는 도메인 (Secrets에 등록 필요)
31     username: ${secrets.EC2_USERNAME} # EC2 접속용 사용자명 (예: ubuntu)
32     key: ${secrets.EC2_SSH_KEY} # SSH 개인 키 (Secrets에 등록)
33     source: "./docker-compose.yml,./dist,./Dockerfile" # EC2로 복사할 파일/폴더 목록
34     target: "~/react-cicd" # EC2 상의 대상 디렉토리
35
36 - name: SSH into EC2 and deploy
37   uses: appleboy/ssh-action@v1.0.3 # EC2 서버에 SSH로 접속하여 명령어 실행
38   with:
39     host: ${secrets.EC2_HOST} # EC2의 IP
40     username: ${secrets.EC2_USERNAME} # 로그인 사용자명
41     key: ${secrets.EC2_SSH_KEY} # SSH 개인 키
42     script: | # 접속 후 실행할 명령어 목록
43       cd ~/react-cicd
44       docker compose down
45       docker compose up -d --build
46

```

docker-compose up -d --build에서 --build 옵션은 컨테이너를 시작하기 전에 이미지를 강제로 새로 빌드(build) 하도록 지정하는 역할

즉, docker-compose.yml가 Dockerfile 기준으로 다시 빌드하라는 뜻

--build가 필요한 이유

docker-compose up 명령어는 기본적으로 다음과 같이 동작

동작	설명	📄
기존 이미지 있음	이전에 빌드된 이미지가 있다면 재사용 함	
코드 변경 감지	자동으로 변경 여부를 감지하지 않음	
결과	코드가 바뀌어도 새로운 이미지 없이 예전 컨테이너 로 실행될 수 있음	

test해기

```
KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/AWS/step20-jwt-aws (main)
$ git add .

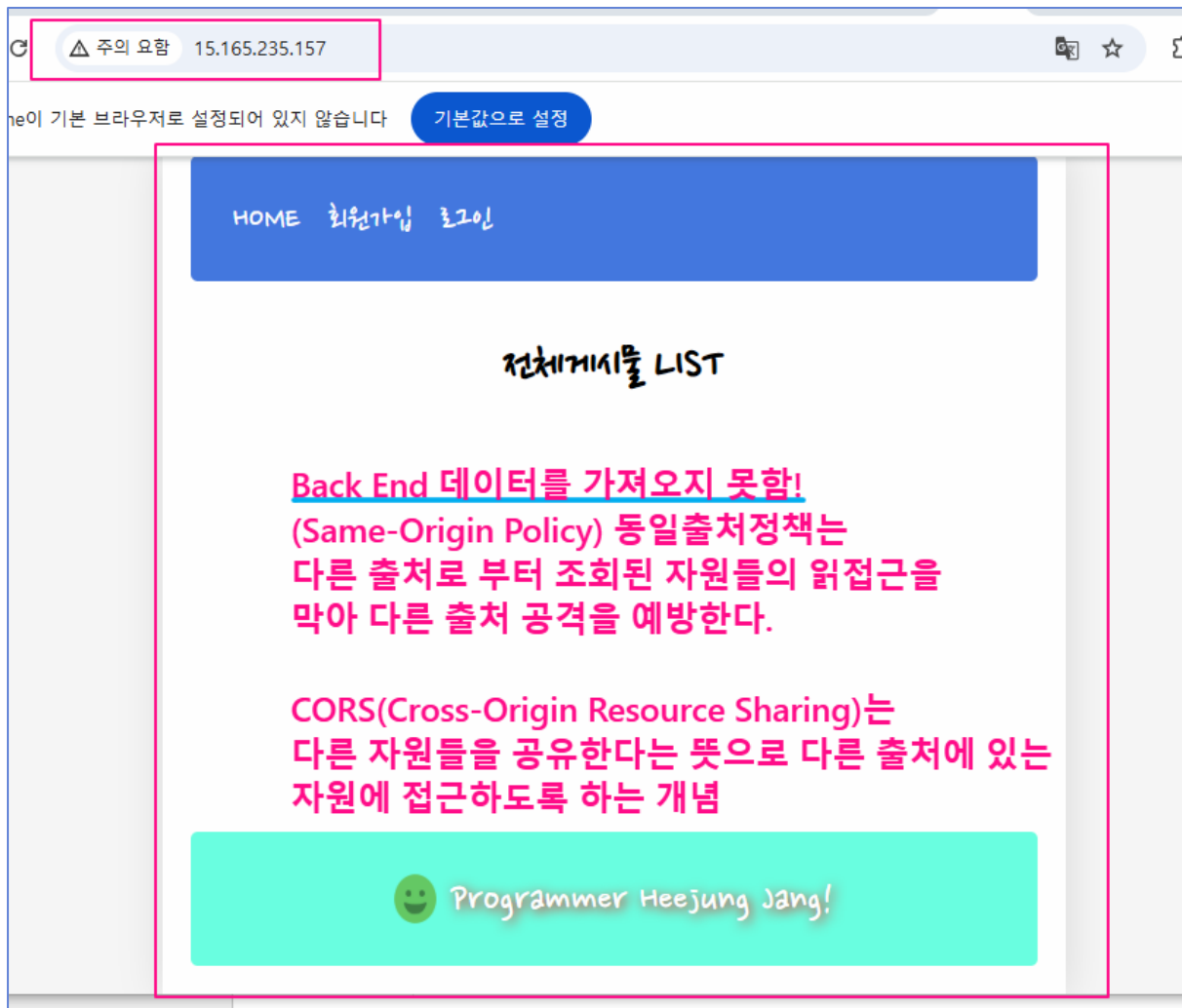
KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/AWS/step20-jwt-aws (main)
$ git commit -m "CI/CD 시작 하기 "
[main fad9630] CI/CD 시작 하기
 2 files changed, 32 insertions(+), 31 deletions(-)

KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/AWS/step20-jwt-aws (main)
$ git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 1.74 KiB | 1.74 MiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/8253jang/cicd03_front_react.git
 0f3f8a8..fad9630  main -> main

KOSTA@DESKTOP-U2D32M7 MINGW64 /d/Edu_286/AWS/step20-jwt-aws (main)
$
```

The screenshot shows the GitHub Actions interface for the repository '8253jang / cicd03_front_react'. The 'Actions' tab is selected, displaying a workflow named 'Deploy React App to EC2' with a job 'CI/CD 시작하기 #4'. The job status is 'succeeded now in 32s'. A list of steps is shown, all marked as successful with green checkmarks:

- > Set up job
- > Build appleboy/scp-action@v0.1.7
- > Build appleboy/ssh-action@v1.0.3
- > Checkout code
- > Setup Node
- > Install dependencies
- > Build React App
- > Copy docker-compose and build folder to EC2
- > SSH into EC2 and deploy
- > Post Setup Node
- > Post Checkout code
- > Complete job



Back-End쪽에 CORS 설정 필요!

web.mvc.config.WebMvcConfig.java

web.mvc.config.SecurityConfig.java

```

11  /**
12   * WebMvcConfigurer 를 이용해서 @CrossOrigin 글로벌 설정
13   * */
14   @Configuration  @BongBong *
15   @EnableWebMvc
16   public class WebMvcConfig implements WebMvcConfigurer {
17       @Override no usages  @BongBong *
18       public void addCorsMappings(CorsRegistry registry) {
19           registry.addMapping(pathPattern: "**")
20           // .allowedOrigins("http://localhost:5173", "http://localhost:4173")
21           // .allowedOrigins("http://43.200.184.113:5173", "http://43.200.184.113:4173")
22           .allowedOrigins("http://15.165.235.157", "http://15.165.235.157:80")
23           .allowedMethods("OPTIONS", "GET", "POST", "PUT", "DELETE");
24       }
25   }

```

```

public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    log.info("SecurityFilterChain filterChain(HttpSecurity http) call.....");
    //CORS 설정
    http.cors(( CorsConfigurer<HttpSecurity> corsCustomizer ->
        corsCustomizer.configurationSource(new CorsConfigurationSource()  @BongBong *
        {
            @Override no usages  @BongBong *
            public CorsConfiguration getCorsConfiguration(HttpServletRequest request) {
                CorsConfiguration configuration = new CorsConfiguration();
                //configuration.setAllowedOrigins(Collections.singletonList("http://localhost:5173"));
                //configuration.setAllowedOrigins(Arrays.asList("http://localhost:5173", "http://localhost:4173"));
                configuration.setAllowedOrigins(Arrays.asList("http://15.165.235.157", "http://15.165.235.157:80"));
                configuration.setAllowedMethods(Collections.singletonList("*"));
                configuration.setAllowCredentials(true);

                configuration.setAllowedHeaders(Collections.singletonList("*"));
                configuration.setMaxAge(3600L);

                configuration.setExposedHeaders(Collections.singletonList("Authorization"));
                return configuration;
            }
        }
    ));
}

```

back-end 수정 > commit > push

